

Compilazione e Linking

Compilazione e Linking

Come funziona il processo di trasformazione del codice C++ in un programma eseguibile.

Come funziona la compilazione?

Quando premi "compila" nel tuo editor, sembra che il programma parta magicamente. In realtà, dietro le quinte succedono tre cose distinte.

Capire questo processo ti aiuta a:

- **interpretare gli errori** — ci sono errori di compilazione ed errori di linking, e si risolvono in modi diversi
- **lavorare con progetti grandi** — con più file `.cpp`, devi sapere come metterli insieme
- **usare il terminale** — spesso si compila da riga di comando, specialmente in ambito professionale

Le tre fasi: dalla ricetta al piatto

Pensa al tuo codice come a una ricetta scritta in italiano. Per "eseguirlo" su un computer, bisogna:

1. **Preprocessing** — il preprocessore prepara gli ingredienti (espande `#include`, sostituisce le macro `#define`, ecc.)
2. **Compilazione** — il compilatore traduce la ricetta in lingua macchina
3. **Linking** — il linker unisce tutti i pezzi in un unico programma eseguibile

Fase 1: Preprocessing

Il preprocessore elabora il tuo codice
prima

della compilazione. Sostituisce ogni `#include` con il contenuto del file incluso, espande le macro e gestisce i blocchi `#ifdef`.

```
#include <iostream>    // viene sostituito con tutto il contenuto di
                        iostream
#define PI 3.14159     // ogni occorrenza di PI viene sostituita con 3.14159

int main() {
    // dopo il preprocessing questa riga diventa:
    // double area = 3.14159 * 5 * 5;
    double area = PI * 5 * 5;
    return 0;
}
```

Puoi vedere il risultato del preprocessing con il flag `-E` :

```
g++ -E main.cpp -o main.i
```

Il file `main.i` contiene il codice dopo il preprocessing — di solito è molto lungo perché include tutto il contenuto degli header.

Fase 2: Compilazione

Il compilatore prende il codice preprocessato e lo trasforma in **codice oggetto** — un file in linguaggio macchina con estensione `.o` (su Linux e macOS) o `.obj` (su Windows).

Il codice oggetto contiene istruzioni che il processore capisce, ma non è ancora un programma completo: mancano i collegamenti alle funzioni scritte in altri file.

```
# Compila senza collegare (-c = solo compilazione)
g++ -c main.cpp -o main.o
g++ -c matematica.cpp -o matematica.o
```

Fase 3: Linking (collegamento)

Il linker prende tutti i file oggetto e li **unisce** in un unico programma eseguibile. Risolve i "ponti" tra i file: se `main.cpp` chiama la funzione `somma()` definita in `matematica.cpp`, il linker collega le due parti.

```
# Collega i file oggetto in un unico eseguibile
g++ main.o matematica.o -o programma
```

Il comando completo con g++

Di solito fai tutto in un unico comando, senza pensare alle fasi:

```
g++ main.cpp matematica.cpp -o programma
```

Questo comando esegue automaticamente tutte e tre le fasi per ogni file `.cpp` e poi collega tutto insieme.

I flag di g++ più utili

I **flag** sono opzioni che puoi aggiungere al comando per modificarne il comportamento. Iniziano sempre con `-`.

Il nome dell'output

```
g++ file.cpp -o mio_programma # l'eseguibile si chiamerà "mio_programma"
g++ file.cpp                  # senza -o, l'eseguibile si chiama "a.out"
```

Warning — gli avvisi del compilatore

I warning non sono errori: il programma compila lo stesso, ma il compilatore ti avvisa di cose sospette.

```
g++ -Wall file.cpp -o output      # abilita la maggior parte dei warning
g++ -Wextra file.cpp -o output    # warning aggiuntivi
g++ -Wall -Wextra file.cpp -o output  # entrambi – consigliato!
```

Esempi di warning utili:

```
int x;           // warning: x usata senza essere inizializzata
if (x = 5)       // warning: probabilmente volevi == invece di =
```

Leggi sempre i warning e correggili — anche se il programma funziona.

Debug — per trovare gli errori

```
g++ -g file.cpp -o output  # include informazioni di debug nel programma
```

Con `-g` puoi usare strumenti come `gdb` per eseguire il programma passo passo e vedere cosa succede.

Ottimizzazione — per rendere il programma più veloce

```
g++ -O0 file.cpp -o output  # nessuna ottimizzazione (utile durante lo sviluppo)
g++ -O1 file.cpp -o output  # ottimizzazione leggera
g++ -O2 file.cpp -o output  # ottimizzazione moderata (consigliata per la versione finale)
g++ -O3 file.cpp -o output  # ottimizzazione aggressiva
```

Usa `-O0` durante lo sviluppo e il debug. Usa `-O2` quando vuoi distribuire il programma.

Standard C++ — scegliere la versione del linguaggio

```
g++ -std=c++11 file.cpp -o output    # C++11
g++ -std=c++14 file.cpp -o output    # C++14
g++ -std=c++17 file.cpp -o output    # C++17 (buona scelta moderna)
g++ -std=c++20 file.cpp -o output    # C++20
```

Specifica sempre lo standard se usi funzionalità moderne. C++17 è una buona scelta oggi:

```
g++ -std=c++17 -Wall -Wextra main.cpp -o programma
```

Aggiungere cartelle per gli header

```
g++ -I./include file.cpp -o output    # cerca gli header anche nella
cartella "include"
g++ -I /percorso/assoluto file.cpp -o output
```

Errori di compilazione vs errori di linking

Saper distinguere i due tipi di errore ti fa trovare il problema molto più in fretta.

Errori di compilazione

Riguardano la **sintassi** o la **logica** del tuo codice. Il compilatore non capisce cosa hai scritto:

```
main.cpp:5:10: error: 'somma' was not declared in this scope
```

Hai usato `somma` senza dichiararla. Manca il prototipo della funzione o l' `#include` dell'header giusto.

```
main.cpp:8:5: error: expected ';' before '}' token
```

Hai dimenticato un punto e virgola.

Errori di linking

Riguardano **riferimenti non risolti** tra file diversi. Il compilatore conosce la funzione, ma il linker non trova la sua definizione:

```
undefined reference to `somma(int, int)'
```

Significa: hai dichiarato `somma` (il compilatore è felice), ma non hai dato al linker il file `.cpp` che la definisce.

Soluzione: aggiungi il file mancante al comando:

```
# SBAGLIATO: manca matematica.cpp
g++ main.cpp -o programma

# CORRETTO
g++ main.cpp matematica.cpp -o programma
```

Esempio pratico con più file

Struttura del progetto:

```
progetto/
├─ main.cpp
├─ studente.h
└─ studente.cpp
```

Compilazione in un unico comando:

```
g++ -std=c++17 -Wall main.cpp studente.cpp -o programma
```

Oppure in due fasi (utile nei progetti grandi — ricompili solo i file modificati):

```
# Compila i singoli file
g++ -std=c++17 -c main.cpp -o main.o
g++ -std=c++17 -c studente.cpp -o studente.o

# Collega tutto
g++ main.o studente.o -o programma
```

Makefile — automatizzare la compilazione

Nei progetti con molti file, riscrivere il comando `g++` ogni volta è noioso e rischioso. Il **Makefile** è un file che descrive come compilare il progetto automaticamente.

```
# Makefile
CXX = g++
CXXFLAGS = -std=c++17 -Wall -Wextra

programma: main.o studente.o
    $(CXX) main.o studente.o -o programma

main.o: main.cpp studente.h
    $(CXX) $(CXXFLAGS) -c main.cpp

studente.o: studente.cpp studente.h
    $(CXX) $(CXXFLAGS) -c studente.cpp

clean:
    rm -f *.o programma
```

Per compilare, scrivi solo:

```
make
```

Per cancellare i file compilati:

```
make clean
```

Non devi padroneggiare il Makefile subito — ma è utile sapere che esiste.

Riepilogo dei flag principali

Flag	Cosa fa
<code>-o nome</code>	Specifica il nome del file di output
<code>-Wall</code>	Abilita i warning principali
<code>-Wextra</code>	Warning aggiuntivi
<code>-g</code>	Aggiunge informazioni per il debug
<code>-O2</code>	Ottimizzazione (consigliata per la versione finale)
<code>-std=c++17</code>	Usa lo standard C++17
<code>-c</code>	Solo compilazione, senza collegamento
<code>-I percorso</code>	Aggiunge una cartella dove cercare gli header
<code>-E</code>	Solo preprocessing

Comando consigliato durante lo studio

Usa sempre questo comando quando studi e sviluppi:

```
g++ -std=c++17 -Wall -Wextra file.cpp -o programma
```

I warning sono tuoi alleati: leggili con attenzione e correggili sempre, anche quando il programma compila senza errori.